THE METHOD · SPEC → MVP

From a spec to a working MVP.

The four-layer build pipeline, walked end-to-end on the real **CoffeeAISystem**. Every layer shown with the actual prototype and the diagram behind it.

4 layers · memory → users → smart → proven

real prototype captures

Context · Component · Activity

Dr. Prasara Jakkaew · ADT MFU

THE WHOLE METHOD ON ONE LINE

Turn the spec into buildable units, then build in four layers

Spec (FR/NFR) → Backlog → Features · MoSCoW → **① Memory** → **② Users + Online** → **③ Smart + Safe** → **④ Proven → MVP**

PHASE 0 · BEFORE YOU BUILD

Make it buildable

- Spec → backlog → **MoSCoW** features
- MVP scope = the **Must-haves** only

LAYERS 1–4 · THE BUILD

Add one capability at a time

- a real **memory** → real **users + a URL**
- an **AI feature with a gate** → **tests that prove it**

An AI agent (Claude Code) assists at every layer; `CLAUDE.md` is the running contract that keeps it aligned with the domain rules.

PHASE 0 · SPEC → BACKLOG → FEATURES

How CoffeeAI became buildable

5 spec docs

FR-01 … FR-42

Marker engine · Analyst Console · LLM narrative + audit · Terroir-omics · Traceability.

1 backlog

NFR-01 … NFR-13

Every FR/NFR in one list, tagged priority (สูง / กลาง / ต่ำ).

12 features

MoSCoW

Clustered into user-meaningful features. สูง→Must, กลาง→Should, ต่ำ→Could.

MVP SCOPE = the **Must-have** features only — about **10 of 12** for CoffeeAI. "Expert Evaluation Rubric" (Could) is deferred out of the MVP. Layers 1–4 build exactly those Must-haves.

THE BUILD · FOUR LAYERS

Each layer gives the prototype one real thing

LAYER	GIVES THE SYSTEM	BUILT WITH	COFFEEAI FEATURE
① **Memory**	a real database	`Firestore`	Lab Ingestion → Sample Library
② **Users + Online**	full data lifecycle, roles, a URL	`CRUD · Auth · Deploy`	Cupping Form · QC Lead vs Roaster
③ **Smart + Safe**	an AI feature with guardrails	`RAG · Audit · Rules`	Cited RAG narrative + Citation Audit
④ **Proven**	tests that verify it	`Playwright`	tests → acceptance criteria

The rest of this deck walks each layer with the **actual CoffeeAI prototype** and the diagram behind it.

LAYER ① · MEMORY

Give it a database — and a domain rule



Prototype · Lab ingest — covers FR-01 / FR-11 / FR-02–08. Rule-based mapping preview (R1–R3), each row explainable.

WHAT YOU DO

- `/init` → **CLAUDE.md**; add the iron rule: **every descriptor carries confidence + supporting_markers + citation**
- Collections: `samples` · `markers` · `mappings` · `cupping_scores` · `reports`
- Ingest writes a real `sample` doc to Firestore

WHY THE RULE MATTERS it is CoffeeAI's DNA. If `CLAUDE.md` omits it, the AI in Layer ③ will happily write ungrounded claims.

LAYER ① · THE DATA MODEL

Five collections — the contract at the centre

COLLECTION	KEY FIELDS
samples	sample_id · origin · process · roast_degree · status · createdAt
markers	marker_id · compound · intensity · ×baseline
mappings	descriptor · confidence · supporting_markers[] · citation · rule
cupping_scores	sample_id · panelist_uid · aroma · acidity · body · sweetness
reports	sample_id · narrative · audit_status · published · role_scope

```
# tell the agent
Model these collections in Firestore.
On mappings, every document MUST carry
  confidence, supporting_markers, citation.
Reject any write that is missing one.
Show me the plan first.
```

THE CONTRACT `mappings` is the one record everything downstream reads. Lock its shape here and the AI in Layer ③ has nothing ungrounded to say.

LAYER ① · PROOF

The list reads live data — and it persists



Prototype · Sample library — FR-10. Every sample with its status: mapped / needs review / queued.

> **THE TEST** ingest a sample → see it on top of the library → **close the browser, reopen → it's still there.** That is the difference between a mockup and a system.

LAYER ② · USERS + ONLINE

Full lifecycle: create · edit · delete



Prototype · Cupping form — FR-12. Panel scores tied to a sample, written to `cupping_scores`.

WHAT YOU DO

- **CRUD** on cupping scores — add, edit, and **delete-with-confirm**
- Firebase **Auth** (email/password); new records store the logged-in `uid`
- Validate-before-save; sort by `createdAt`

NOTE cupping data is research evidence — **delete must always confirm**. No silent loss.

LAYER ② · ACCESS CONTROL (ACL.MD)

Who can do what — and what they cannot

ROLE	CAN	CANNOT
QC / Cupping Lead	ingest · map · cup · publish reports	–
Cupping Panel	enter own cupping scores	see/edit others' scores · publish
Roaster / Buyer	read published Public Report	any write · see provenance PII
not signed in	read published reports only	everything else

RULE OF THUMB the **Cannot** column is the one that matters — the AI's first draft usually leaves it empty. Fill it now; it becomes the **Firestore rules** in Layer ③, word for word.

LAYER ② · ROLES + A LIVE URL

Same system, different role, different view



Prototype · Public report — FR-19. Read-only surface for a roaster / buyer, same design system.

ACL.MD → UI

- **QC / Cupping Lead:** ingest, cup, edit mapping status
- **Roaster / Buyer:** read Public Report **only** — no edit, no delete
- Deploy → a real `*.web.app` URL a buyer can open

HONEST LIMIT hiding buttons is **not** real security yet — the real door is the Firestore rules in Layer ③. This `ACL.md` is reused there.

LAYER ③ · SMART + SAFE · THE STAR OF COFFEEAI

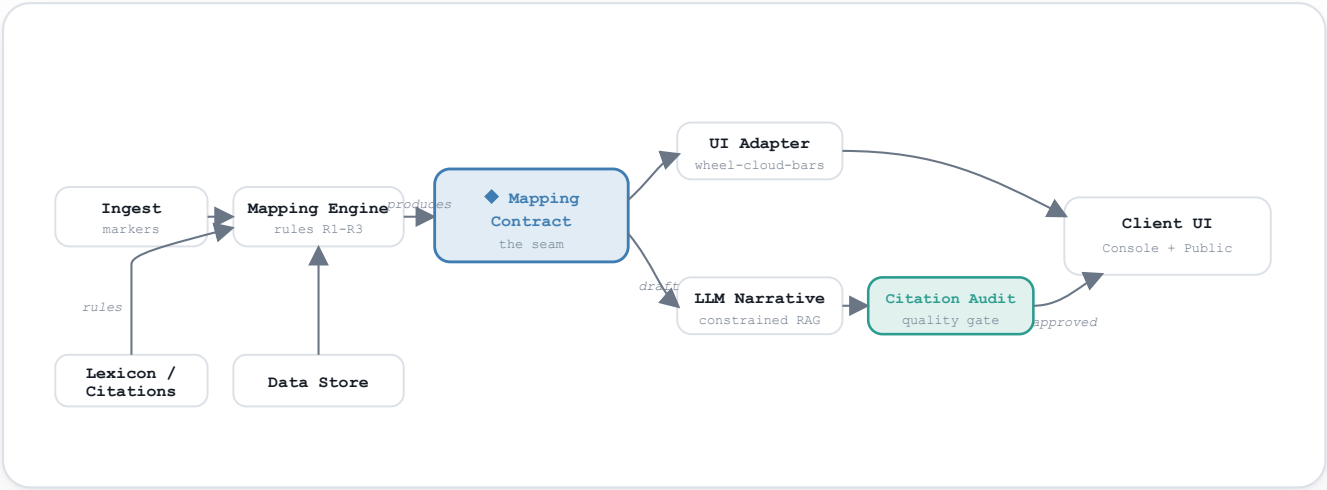
An AI narrative that must cite its evidence



Prototype · LLM narrative — FR-20–28. Constrained RAG: the model sees **only the mapping contract**; every claim carries a citation [1][2][3]; unsure claims are tagged **speculative**.

LAYER ③ · THE ARCHITECTURE BEHIND IT (C4 COMPONENT)

A seam the LLM can't cross — and a gate it can't skip



THE SEAM downstream sees **only** `descriptor · confidence · supporting_markers · citation` → the LLM **cannot invent chemistry it never receives**. The **Citation Audit** is the NFR "every claim cited" turned into a component. Firestore rules enforce the `ACL.md` at the database — the real door.

LAYER ③ · THE REAL DOOR · FIRESTORE RULES

Security by structure, not hidden buttons

```
rules_version = '2';
service cloud.firestore {
  match /databases/{db}/documents {

    // published reports: world-readable
    match /reports/{id} {
      allow read:  if resource.data.published == true;
      allow write: if isRole('qc_lead');
    }
    // samples & mappings: QC writes only
    match /samples/{id} {
      allow read:  if signedIn();
      allow write: if isRole('qc_lead');
    }
    // cupping: panellist edits only own row
    match /cupping_scores/{id} {
      allow write: if request.auth.uid
                   == request.resource.data.panelist_uid;
    }
  }
}
```

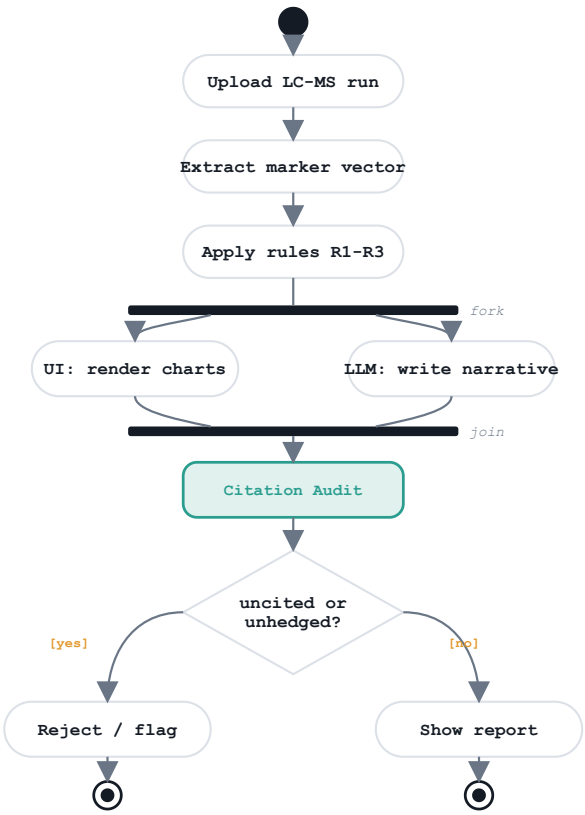
WHY THIS BEATS "HIDE THE BUTTON"

- a request with **no valid role** is refused **at the database**
- bypassing your web UI changes nothing
- PDPA: provenance PII stays on a QC-only path

TEST IT sign in as `roaster` → attempt any write → **PERMISSION_DENIED**. The same `ACL.md` from Layer ② is now enforced.

LAYER ③ · THE WORKFLOW (UML ACTIVITY)

The gate blocks a bad report automatically



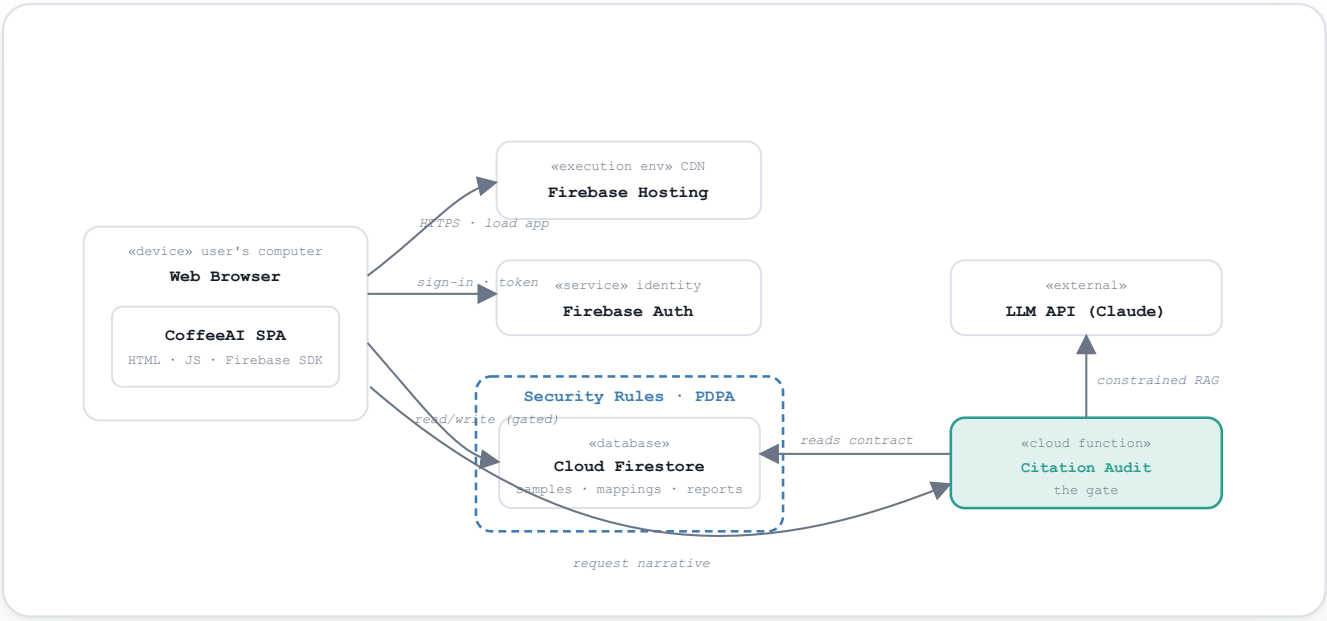
READ THE FLOW

- chemistry in → rules → **fork**: UI + LLM run in parallel
- **join** → the **Citation Audit** gate
- a claim uncited or unhedged → **[yes] reject/flag**; otherwise **[no] show report**

AI CHECKS AI a second agent audits the first agent's output and **blocks publishing** until it complies. That is agentic safety made structural.

HOW IT RUNS · UML DEPLOYMENT

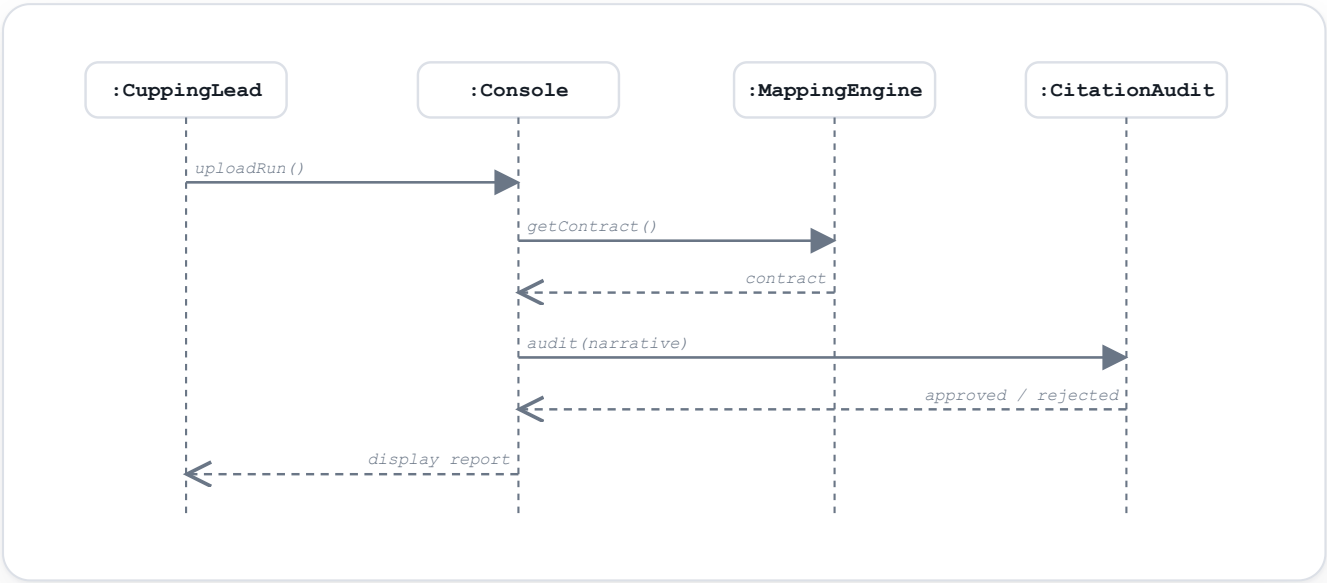
Where each part lives — and how they talk



READ IT the browser only ever talks **HTTPS**; Firestore sits **inside the Security-Rules boundary** (nothing writes without a valid role); the LLM is reached **only** through the **Citation Audit** function — so no unaudited narrative can reach a user. Swap the LLM vendor without touching anything else.

MODELLING VIEW · UML SEQUENCE

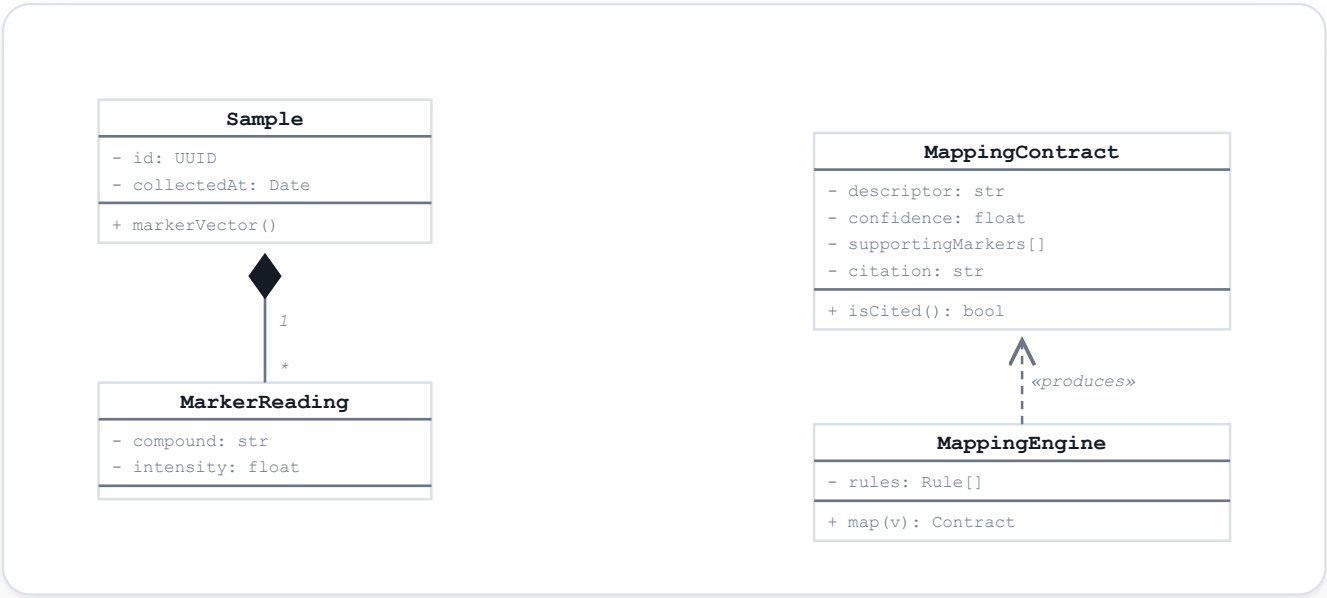
The message order — time flows down



READ IT each dashed vertical is a **lifeline**; a solid arrow is a **call**, a dashed arrow a **return**. Top-to-bottom: CuppingLead uploads a run → Console fetches the **contract** → the narrative is **audited** before the report is shown. Midterm D5.

MODELLING VIEW · UML CLASS

The structure — the MappingContract at the core



READ IT three compartments per class — **name / attributes / methods**. Filled ◆ = **composition** (a Sample owns its MarkerReadings, **1 … ***); dashed arrow = **dependency** («produces»). **MappingContract** carries `confidence + citation` and `isCited()` — exactly what the audit checks. Midterm D6.

LAYER ④ · PROVEN

Tests that trace back to the spec



Prototype · Sample report — FR-13–18. Radar · flavour wheel · word cloud · intensity bars, from the same contract.

PLAYWRIGHT · 3 CHECKS

- ingest → mapping has **confidence + citation** (FR-02–05)
- generated narrative has **0 uncited claims** (audit passes)
- Public Report is **read-only** for a roaster

CLOSE map each test to an acceptance criterion in `test-report.md` → every Must-have feature is provably covered. **MVP done.**

LAYER ④ · TESTS TRACED TO THE SPEC

Every Must-have has a test that names its FR

```
// tests/mvp.spec.ts
test('FR-02..05 · mapping is grounded', async ({ page }) => {
  await page.goto('/sample-report?id=SMP-562');
  const row = page.locator('[data-testid=mapping-row]').first();
  await expect(row).toContainText('confidence');
  await expect(page.locator('.citation')).not.toHaveCount(0);
});

test('FR-26 · no uncited claim', async ({ page }) => {
  await page.goto('/llm-narrative?id=SMP-562');
  await expect(page.locator('.audit-status')).toHaveText('passed');
});
```

TEST	PROVES
mapping grounded	FR-02–05 · NFR-01
no uncited claim	FR-26–28 · NFR-08
public read-only	FR-19 · rules

MVP DONE green suite + a `test-report.md` that maps each test to an acceptance criterion = every Must-have is **provably** covered, not just demoed.

RECAP · WHY THIS CASE EARNS ITS KEEP

CoffeeAI is a richer case than plain CRUD

GENERIC HELPER ✕

"AI writes a summary"

- No guarantee it's grounded
- Nothing stops a confident hallucination

COFFEEAI ✓

Cited + audited narrative

- A **seam** feeds the LLM only cited evidence
- A **gate** blocks any uncited claim
- Layers ③–④ carry real academic weight: citation integrity + PDPA traceability

① Memory → ② Users+Online → ③ Smart+Safe → **④ Proven → MVP**

CLOSE · YOUR MVP CHECKLIST

Build the Must-haves, one layer at a time

Phase 0 · spec → backlog → MoSCoW → prototype

① repo + **CLAUDE.md** (citation rule) + Firestore ingestion

② CRUD + Auth roles + a live URL

③ cited RAG narrative + Citation Audit + Firestore rules

④ Playwright passing, traced to acceptance criteria

Case: **CoffeeAISystem** (`raw/28_CoffeeAISystem`). Prototype captures are the real `01-prototypes/20260816-01-v1` screens. AI-First · Vibe Coding · Agentic throughout.

MIDTERM REVIEW · GUIDELINE

Midterm — read the case, then decide.

A study guide for the 1305493 midterm. It covers **Weeks 1–6**; the diagram block is exactly the modelling you practised on CoffeeAI.

60 items · case-study | 6 mini-cases A–F | CLO3 diagrams = biggest block

MIDTERM · FORMAT

What the paper looks like — and how to read it

THE PAPER

- **60 items × 1 point** = 60
- **Case-study analysis** — 6 mini-cases (A–F)
- Covers **Weeks 1–6** only
- Every item asks you to **diagnose / evaluate / prioritise / read a diagram** — not recall a definition

HOW TO READ AN ITEM

- Read the **scenario first**, then the question
- Underline the **role / constraint** that decides the answer
- Watch **reverse items** ("which is **NOT**…")
- No "all of the above"; A/B/C/D balanced — **length is not a tell**

MIDTERM · COVERAGE MAP

Six cases · where the marks are

CASE	SCENARIO	WEEK	ITEMS	FOCUS
A	BrewBuddy — starting up	W1	1–8	engineer's value, guardrails, metric
B	MediQueue — AI symptom checker	W2	9–21	AI ethics, bias, **PDPA / CCA / ETDA**
C	ShopSense — notes → backlog	W3	22–31	requirement vs solution, user story, agents
D	FitTrack — design before building	W4	32–45	**the diagrams (what you drew on CoffeeAI)**
E	GreenCart — at the gate	W5	46–52	gate pass/fail, traceability, evidence
F	LinguaLoop — scope lock, Sprint 0	W6	53–60	MoSCoW, WON'T, Sprint 0, MCP, AI governance

CLO3 (design & diagrams) = 19 of 60 — the single biggest block. CLO1 (quality / ethics / PDPA) 15 · CLO2 13 · CLO5 (AI tools / Git / MCP) 8. The three assessed CLOs carry ~78%.

Case D · 14 items — all diagrams

DIAGRAM	UML / C4	ANSWERS THE QUESTION
Context	C4 L1	who / what does the system talk to?
Use-Case	UML	what goals do actors achieve? («include»/«extend»)
Component	C4 L3	what are the parts + the **seam**?
Activity	UML	how does the workflow flow + branch? (guard, fork/join)
Sequence	UML	message order over time (lifelines, call/return)
Class	UML	structure: attributes, methods, multiplicity, composition

STUDY MOVE if you can **draw** these for CoffeeAI, you can **read** them on FitTrack in the exam. This deck draws **Component, Activity, Sequence & Class** for CoffeeAI (plus a Deployment view); **Context** & **Use-Case** are in the W7 architecture deck.

MIDTERM · FACTS TO LOCK + TIPS

The legal facts examiners love

PDPA

Lawful basis before collecting; **health = sensitive data** (stricter); consent + purpose limitation.

CCA §26

Keep traffic logs **90 days**; penalty up to **500,000 baht**.

ETDA · ETA

6 ETDA principles; ETA §9 e-signature test, §28 CA duties.

AI governance

Disclose AI use; a human stays accountable (the Air Canada parallel).

EXAM TIPS answer the **case**, not a memorised definition · reverse items exist ("which is **NOT**…") · A/B/C/D each appear ~15× — don't pattern-guess · for diagram items, name the **notation** (initial node, guard, seam, multiplicity).

APPENDIX A · DEFINITION OF DONE

Closing checklist — tick every box

PHASE 0 · BUILDABLE

- ☐ spec → backlog (FR/NFR)
- ☐ 12 features tagged MoSCoW
- ☐ MVP scope = Must-haves only

① MEMORY

- ☐ repo pushed · no key files
- ☐ `CLAUDE.md` has the citation rule
- ☐ `rule.md` lists the NFR guardrails
- ☐ ingest writes real Firestore doc
- ☐ close browser → data persists

② USERS + ONLINE

- ☐ create · edit · delete-with-confirm
- ☐ Auth; records store `uid`
- ☐ `ACL.md` "Cannot" column filled
- ☐ live `*.web.app` URL

③ SMART + SAFE

- ☐ narrative — every claim cited
- ☐ Citation Audit blocks bad output
- ☐ Firestore rules deny `roaster` writes
- ☐ reviewer pass committed

④ PROVEN

- ☐ Playwright suite green
- ☐ each test names an FR
- ☐ `test-report.md` written

SHIP

- ☐ every Must-have demoed on the URL
- ☐ `KNOWN-GAPS.md` lists what's stubbed
- ☐ submit: repo URL + live URL

APPENDIX B · RULE.MD — THE AGENT'S GUARDRAIL FILE

One file of NFRs the agent must obey

```
# rule.md — CoffeeAI guardrails (NFR)


R1  Every descriptor carries confidence +
    supporting_markers + citation. No uncited claim.
R2  Speculative claims must be hedged (tag speculative).
R3  Mapping engine is rule-based & explainable —
    no black-box model.              (NFR-01)
R4  No provenance PII on any public surface. (PDPA)
R5  Nothing publishes until Citation Audit passes.
```

HOW TO USE IT

- the agent **reads it before every change**
- load-bearing rules become **gates** (Citation Audit) & **Firestore rules**
- can't meet a rule yet? log it in `KNOWN-GAPS.md` — don't hide it

```
Create rule.md with the NFR guardrails above.
From now on, read rule.md before every change and
keep the build compliant. If a rule can't be met
yet, log it in KNOWN-GAPS.md instead of hiding it.
```

THREE FILES, THREE JOBS `CLAUDE.md` = project brief for the AI · `ACL.md` = who can do what · `rule.md` = the NFR guardrails that turn into the **seam**, the **audit gate**, and the **security rules**.

APPENDIX C · PROMPT PLAYBOOK · LAYER ①

Give it a memory (real lab commands, adapted)

CLAUDE.MD · DOMAIN RULE

```
Fill in CLAUDE.md with 3 things:
all Firestore collection names, the 3 sample
statuses (mapped / needs review / queued), and
the iron rule — every descriptor carries
confidence + supporting_markers + citation;
never commit keys.
```

CONNECT FIRESTORE

```
Here is my firebaseConfig: <paste>.
Write a plan to connect the CoffeeAI web app
to Firestore.
Show me the plan first — don't start yet.
```

SEED SAMPLE DATA

```
Write a script to seed data into Firestore:
- samples: 5 docs (sample_id, origin, process,
  roast_degree, status, createdAt)
- markers, mappings, cupping_scores to match
Write the plan for me to review, then wait for my go.
```

READ FROM FIRESTORE

```
The Sample Library page still uses mock data
in the file. I want it to read from the
samples collection in Firestore instead.
Show me the plan first — don't start yet.
```

APPENDIX D · PROMPT PLAYBOOK · LAYER ②

Users, roles & a live URL

FORM → FIRESTORE

```
I want the Cupping Form to actually save to
Firestore, tied to sample_id. Set the sample's
initial status to "needs review", then return
to the list. Show me the plan first — don't start yet.
```

EDIT ONLY THE STATUS FIELD

```
On the sample detail page I want buttons to set
the mapping status. When clicked, change ONLY the
status field — touch nothing else.
Show me the plan first.
```

DELETE WITH CONFIRM

```
Add a delete button for a cupping score, but it
must always ask to confirm first. If I cancel,
nothing is deleted. Show me the plan first.
```

EMAIL / PASSWORD LOGIN

```
I want email/password login with Firebase.
Show me the plan first — don't start yet.
```

ACL.MD · DRAFT → FILL → ENFORCE

```
CoffeeAI roles: qc_lead ingests/maps/cups/
publishes, cupping_panel enters own scores,
roaster reads the public report.
Draft ACL.md — columns: role / can / cannot.
— then —
It's missing that a roaster cannot write or see
provenance PII, and a panellist cannot see
others' scores. Fill them in, overwrite the file.
— then —
```

Read ACL.md and limit on-screen buttons/menus to

DEPLOY

I want to deploy this to Firebase Hosting

together with the rules.

Give me one command at a time and tell me

what I'll see on screen.

Smart & safe — the gate

FIRESTORE SECURITY RULES

```
Write Firestore Security Rules to match ACL.md:
roaster + anonymous read published reports only;
qc_lead writes samples/mappings; a panellist
edits only their own cupping row.
Write the rules file for me to read first —
don't deploy yet.
```

CONSTRAINED-RAG NARRATIVE

```
I want a button "generate the sensory narrative".
Step 1: read ONLY the mapping contract for this
sample (descriptor, supporting_markers, confidence,
citation). Step 2: write a Consumer and a Roaster
narrative — every claim needs a [citation], tag any
guess as speculative. Step 3: save it to the report.
Show me the plan first — don't start yet.
```

CITATION AUDIT GATE + LOG

```
Add an automatic Citation Audit before publish:
scan the narrative for (1) uncited claims,
(2) speculative claims with no hedge. Log input,
output, createdAt to an auditLog sub-collection
each run. Never publish automatically — block until
it passes and a human confirms.
Do it one item at a time; report each before moving on.
```

MOVE KEYS OUT BEFORE PUSH

```
I'm about to push to GitHub. Move the AI key out
of the code into a separate file that .gitignore
excludes, then show me the list of files about to
go to GitHub.
```

APPENDIX F · PROMPT PLAYBOOK · LAYER ④

Prove it — tests

TESTER SUB-AGENT (PLAYWRIGHT MCP)

```
Create a sub-agent called "tester" (model sonnet).
Its job: use Playwright MCP to open a real browser
and test in place of a human, following the CoffeeAI
spec. Don't change the system's code to make tests
pass. Let me read the file first.
```

GROUNDING + AUDIT TEST

```
Add tests: open a sample report — the mapping must
show confidence and at least one citation; and the
generated narrative's audit status must be "passed".
```

VALIDATION + SECURITY TESTS

```
Add a test: submit the cupping form with the sample
field blank and save — it must not save and must
show a message.
Add 2 security tests: (1) not logged in, open the
list → cannot read; (2) as a roaster, edit a report
→ must fail. Both pass when access is denied.
```

RUN ALL + REPORT

```
Run all tests again and summarize into
test-report.md: what each test checks, pass/fail,
and when it ran. For failures, say where it's stuck.
```

The whole playbook, in order — one prompt sequence:

rule.md + CLAUDE.md → connect Firestore → CRUD + login + ACL → deploy → security rules → **cited narrative** → **Citation Audit** → Playwright tests → **test-report → MVP**

HABIT these follow the lab's own command style — end build prompts with **"show me the plan first, don't start yet"**, read the plan, add what's missing, then let it run.